

EP03 - ULA DE 4 BITS + SIMULADOR ARDUINO

Disciplina: Arquitetura de Computadores II **Data:** Outubro / 2026

Aluno: Arthur De Pinho De Almeida **Matrícula:** 888678

Instituição: Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

1. Visão Geral da Arquitetura do Sistema

O Exercício Prático 03 (EP03) consiste no projeto e implementação de um ecossistema completo para simulação de uma **Unidade Lógica e Aritmética (ULA) de 4 bits** composta por dois módulos principais:

- 1. Compilador em C++ (PC):** Lê um programa fonte em mnemônicos (`.ula`), valida sintaxe, reporta erros linha a linha, gera o código de máquina em formato hexadecimal (`.hex`) e fornece a string concatenada de triplas `xys`.
- 2. Simulador no Arduino (Hardware/Tinkercad):** Recebe a string de instruções via porta serial, gerencia um vetor de memória interna de 100 posições, executa o ciclo de instrução (Busca, Decodificação, Execução, Escrita), aciona 4 LEDs físicos de saída e imprime o Dump formatado da memória.

Fluxo de Dados:

Arquivo `.ula` → Compilador C++ → String Hex (ex: `C6BA3E`) → Monitor Serial

Arduino → Memória RAM (100 posições) → ULA 4 bits → LEDs + Dump Serial

2. Parte 1: Explicação Detalhada do Compilador

(`compilador.cpp`)

O objetivo do compilador é traduzir instruções textuais de alto nível em instrução de máquina de 3 nibbles (12 bits totais por instrução), onde cada instrução segue o formato **XYS**:

- **X (4 bits):** Valor do operando A (0 a F)
- **Y (4 bits):** Valor do operando B (0 a F)
- **S (4 bits):** Seletor da Operação Lógica da ULA (0 a F)

2.1 Tabela de Opcodes (Figura 2 do EP03)

Opcode (S)	Mnemônico	Função Lógica	Descrição
0	nA	A'	NOT A (Inverte A)
1	AoBn	(A+B)'	NOR(A, B)
2	nAeB	A'.B	NOT(A) AND B
3	zeroL	0	Constante Zero (0000)
4	AeBn	(A.B)'	NAND(A, B)
5	nB	B'	NOT B (Inverte B)
6	AxB	A ⊕ B	XOR(A, B)
7	AenB	A.B'	A AND NOT(B)
8	nAoB	A' + B	NOT(A) OR B

Opcode (S)	Mnemônico	Função Lógica	Descrição
------------	-----------	---------------	-----------

9	AxBn	$(A \oplus B)'$	XNOR(A, B)
A	copiaB	B	Passa valor de B
B	AeB	A.B	AND(A, B)
C	umL	1 (15)	Constante Um (1111)
D	AonB	$A + B'$	A OR NOT(B)
E	AoB	$A + B$	OR(A, B)
F	copiaA	A	Passa valor de A

2.2 Código do Compilador Comentado Passo a Passo

A) Estruturas de Dados e Utilitários

```
// Mapeia mnemônico -> caractere hexadecimal do opcode
const map<string, char> OPCODES = {
    {"nA", '0'}, {"AoBn", '1'}, {"nAeB", '2'}, {"zeroL", '3'},
    {"AeBn", '4'}, {"nB", '5'}, {"AxB", '6'}, {"AenB", '7'},
    {"nAoB", '8'}, {"AxBn", '9'}, {"copiaB", 'A'}, {"AeB", 'B'},
    {"umL", 'C'}, {"AonB", 'D'}, {"AoB", 'E'}, {"copiaA", 'F'}
};

// Converte caractere para maiúsculo (permite aceitar x=, y=, w= minúsculos)
string trim(const string& str) {
    size_t first = str.find_first_not_of(" \t\r\n");
    if (first == string::npos) return "";
    size_t last = str.find_last_not_of(" \t\r\n");
    return str.substr(first, last - first + 1);
}

// Converte inteiro (0-15) em caractere Hexadecimal ('0'-'F')
char paraHexa(int val) {
    val &= 0x0F;
    return (val < 10) ? ('0' + val) : ('A' + val - 10);
}
```

```
}
```

B) O comando `transform(... ::toupper)`

```
transform(varUpper.begin(), varUpper.end(), varUpper.begin(), ::toupper);
```

Explicação Didática: Essa função percorre cada caractere da string `varUpper` (do

início `begin()` até o fim `end()`) e aplica a função C/C++ `::toupper`, sobrescrevendo o

resultado na própria variável. Isso garante insensibilidade ao caso (case-insensitivity),

permitindo que o usuário escreva `x = 12` ou `X = 12` indiferentemente.

C) Análise Sintática (`processarLinha`)

```
bool processarLinha(const string& linhaOriginal, int numLinha,
                   int& valorX, int& valorY,
                   vector<string>& instrucoes) {
    string linha = trim(linhaOriginal);

    // 1. Tratamento de Erro: Linha em branco
    if (linha.empty()) {
        cout << " -> [ERRO - Linha " << setw(3) << numLinha
              << "]: Linha em branco detectada (ignorada na carga)." << endl;
        return false;
    }

    // 2. Ignora marcadores de bloco sem gerar erro
    if (linha == "inicio:" || linha == "fim.") return true;

    // Remove ponto e vírgula no final se houver
    if (linha.back() == ';') linha.pop_back();

    // 3. Separação por '='
    size_t posIgual = linha.find('=');
```

```

    if (posIgual == string::npos || posIgual == 0) {
        cout << " -> [ERRO - Linha " << setw(3) << numLinha
            << "]: Sintaxe invalida (sem '='): '" << linha << "'" << endl;
        return false;
    }

    string variavel = trim(linha.substr(0, posIgual));
    string valor     = trim(linha.substr(posIgual + 1));
    string varUpper = variavel;
    transform(varUpper.begin(), varUpper.end(), varUpper.begin(), ::toupper);

    // 4. Processamento das atribuições X, Y e W
    if (varUpper == "X") {
        try {
            valorX = parseValor(valor) & 0x0F; // Trunca em 4 bits (0-15)
            cout << " -> [OK - Linha " << setw(3) << numLinha
                << "]: X = " << valorX << " (Hex: " << paraHexa(valorX) << ")" <<
endl;
        } catch (...) {
            cout << " -> [ERRO - Linha " << setw(3) << numLinha
                << "]: Valor invalido para X: '" << valor << "'" << endl;
            return false;
        }
    } else if (varUpper == "Y") {
        try {
            valorY = parseValor(valor) & 0x0F;
            cout << " -> [OK - Linha " << setw(3) << numLinha
                << "]: Y = " << valorY << " (Hex: " << paraHexa(valorY) << ")" <<
endl;
        } catch (...) {
            cout << " -> [ERRO - Linha " << setw(3) << numLinha
                << "]: Valor invalido para Y: '" << valor << "'" << endl;
            return false;
        }
    } else if (varUpper == "W") {
        auto it = OPCODES.find(valor);
        if (it != OPCODES.end()) {
            // Monta a tripla XYZ
            char hexX = paraHexa(valorX);
            char hexY = paraHexa(valorY);
            char hexS = it->second;
            string instrucao = "";
            instrucao += hexX;
            instrucao += hexY;
            instrucao += hexS;
            instrucoes.push_back(instrucao);
        }
    }
}

```

```

        cout << " -> [OK    - Linha " << setw(3) << numLinha
        << "]: W = " << valor << " -> Instrucao gerada: " << instrucao <<
endl;

    } else {
        cout << " -> [ERRO  - Linha " << setw(3) << numLinha
        << "]: Mnemonico invalido: '" << valor << "'" << endl;
        return false;
    }
} else {
    cout << " -> [ERRO  - Linha " << setw(3) << numLinha
    << "]: Variavel desconhecida '" << variavel << "'" << endl;
    return false;
}

return true;
}

```

2.3 Exemplo Completo de Execução do Compilador

Arquivo de Entrada (**testeula.ula**):

```

inicio:
X = 12
Y = 6
W = AeB
X = 10
Y = 3
W = AoB
fim.

```

Passo a Passo da Tradução:

1. **X = 12** → Registrador \$X = 12\$ (Hexadecimal **C**)
2. **Y = 6** → Registrador \$Y = 6\$ (Hexadecimal **6**)
3. **W = AeB** → Opcode de **AeB** = **B**. Tripla gerada: **C6B**
4. **X = 10** → Registrador \$X = 10\$ (Hexadecimal **A**)
5. **Y = 3** → Registrador \$Y = 3\$ (Hexadecimal **3**)
6. **W = AoB** → Opcode de **AoB** = **E**. Tripla gerada: **A3E**

String Hexadecimal Final Gerada: **C6BA3E**

3. Parte 2: Explicação Detalhada do Arduino (`arduino\_ula.ino`)

3.1 Estrutura de Memória RAM Simulada

```
String memoria[100]; // Vetor de 100 posições da memória do simulador

// Mapeamento das Posições:
// memoria[0] = PC (Program Counter) -> Inicia no índice 4
// memoria[1] = W (Resultado da última operação)
// memoria[2] = X (Operando A)
// memoria[3] = Y (Operando B)
// memoria[4..99] = Área de instruções do programa carregado
```

3.2 Carga de Memória e Leitura de Triplas (`load\_memory`)

```
void load_memory(String input) {
    // Reseta todo o vetor para eliminar resíduos de execuções anteriores
    for (int i = 0; i < 100; i++) {
        memoria[i] = "";
    }

    pc = 4;
    memoria[0] = String(pc); // PC aponta para 4
    memoria[1] = "0";        // W = 0
    memoria[2] = "0";        // X = 0
    memoria[3] = "0";        // Y = 0
    n = 0;
    pos_input = 0;

    // Carrega as triplas XYS a partir do índice 4
    while (pos_input < input.length() && n + 4 < 100) {
        String sub = get_sub_string(input);
        if (sub.length() < 3) break; // Interrompe se não houver 3 caracteres válidos
        memoria[n + 4] = sub;
        n++;
    }

    Serial.println("\n[+] Carga do vetor realizada!");
    display_memoria();
}

String get_sub_string(String input) {
    int j = 0;
```

```

String sub = "";
while (j < 3 && pos_input < input.length()) {
    char c = input.charAt(pos_input);
    if (c != ' ' && c != '\n' && c != '\r') {
        sub += c;
        j++;
    }
    pos_input++;
}
return sub;
}

```

Destaque de Arquitetura: A checagem `if (sub.length() < 3) break;` evita que

caracteres de controle ou quebras de linha no final da string criem instruções

inválidas e façam o controlador executar posições vazias até o índice 99.

3.3 Ciclo de Busca e Execução (`program\_execution`)

```

void program_execution() {
    Serial.println("\nQuer executar o programa? (s/n)");
    while (!Serial.available()) { delay(100); }
    String resp = Serial.readString();
    resp.trim();

    if (!resp.equalsIgnoreCase("s") && !resp.equalsIgnoreCase("sim")) return;

    for (int i = 0; i < n; i++) {
        String instrucao = memoria[pc];
        if (instrucao.length() < 3) break;

        // Decodificação dos nibbles
        char charX = instrucao.charAt(0);
        char charY = instrucao.charAt(1);
        char charS = instrucao.charAt(2);

        // Execução da operação na ULA
    }
}

```

```

    int resultadoInt = executar_ula(charX, charY, charS);

    // Escrita nos registradores
    W = converter_para_hexa(resultadoInt);
    regX = charX;
    regY = charY;

    // Atualização dos LEDs e exibição do Dump com a seta ->
    acionar_leds(resultadoInt);
    display_memoria_com_pc(pc);

    pc++;
    delay(4000); // Intervalo de 4 segundos entre instruções
}

Serial.println("\n=== Execucao finalizada! ===\n");
}

```

3.4 O Núcleo Lógico da ULA de 4 Bits (`executar\_ula`)

```

int executar_ula(char x, char y, char s) {
    int A = converter_para_int(x);
    int B = converter_para_int(y);
    int resultado = 0;

    switch (s) {
        case '0': resultado = (~A);      break; // nA      : A'
        case '1': resultado = ~(A | B);  break; // AoBn    : (A+B)'
        case '2': resultado = (~A) & B;  break; // nAeB     : A'.B
        case '3': resultado = 0;          break; // zeroL    : 0
        case '4': resultado = ~(A & B);  break; // AeBn     : (A.B)'
        case '5': resultado = (~B);      break; // nB       : B'
        case '6': resultado = A ^ B;      break; // AxB      : XOR
        case '7': resultado = A & (~B);   break; // AenB     : A.B'
        case '8': resultado = (~A) | B;   break; // nAoB     : A' + B
        case '9': resultado = ~(A ^ B);   break; // AxBn     : XNOR
        case 'A': resultado = B;          break; // copiaB   : B
        case 'B': resultado = A & B;      break; // AeB      : A.B
        case 'C': resultado = 0x0F;       break; // umL      : 1 (15 em 4 bits)
        case 'D': resultado = A | (~B);   break; // AonB     : A + B'
        case 'E': resultado = A | B;      break; // AoB      : A + B
        case 'F': resultado = A;          break; // copiaA   : A
    }

    return resultado & 0x0F; // Trunca e garante que o resultado é de 4 bits
}

```

```
}
```

3.5 Controle Físico dos LEDs (`acionar\_leds`)

```
void acionar_leds(int valorW) {  
    valorW &= 0x0F;  
    digitalWrite(PINO_F3, (valorW & 0x08) ? HIGH : LOW); // Bit 3 (MSB) -> Pino 13  
    digitalWrite(PINO_F2, (valorW & 0x04) ? HIGH : LOW); // Bit 2      -> Pino 12  
    digitalWrite(PINO_F1, (valorW & 0x02) ? HIGH : LOW); // Bit 1      -> Pino 11  
    digitalWrite(PINO_F0, (valorW & 0x01) ? HIGH : LOW); // Bit 0 (LSB) -> Pino 10  
}
```

A função aplica ****máscaras bitwise**** para isolar cada um dos 4 bits de \$W\$:

- Bit 3 (\$8_{10} = 1000_2\$): Pino 13
- Bit 2 (\$4_{10} = 0100_2\$): Pino 12
- Bit 1 (\$2_{10} = 0010_2\$): Pino 11
- Bit 0 (\$1_{10} = 0001_2\$): Pino 10

3.6 Impressão do Dump no Monitor Serial

```
void display_memoria_com_pc(int address) {  
    Serial.print(" | ");  
    Serial.print(pc);    Serial.print(" | ");  
    Serial.print(W);     Serial.print(" | ");  
    Serial.print(regX);  Serial.print(" | ");  
    Serial.print(regY);  Serial.print(" | ");  
    for (int i = 4; i < n + 4; i++) {  
        if (i == address) {  
            Serial.print("->");  
        }  
        Serial.print(memoria[i]);  
        Serial.print(" | ");  
    }  
    Serial.println();  
}
```

Otimização de Desempenho (SRAM): O uso de chamadas diretas a `Serial.print()`

evita o estouro de memória RAM (SRAM de 2KB do Uno) que ocorre quando se usa

concatenação excessiva da classe `String`.

4. Guia de Roteiro para Apresentação ao Professor

1. **Apresente a Arquitetura Geral:** Explique que o projeto é dividido entre o Compilador (Software em C++) e a ULA Simulada (Hardware no Arduino com 4 LEDs).
2. **Mostre a Compilação:** Execute `./compilador testeula.ula testeula.hex` no terminal e mostre como as linhas de atribuição e mnemônicos viram triplas hexadecimais \$XYS\$.
3. **Mostre o Carregamento no Tinkercad:** Cole a string gerada no Monitor Serial do Arduino e destaque o primeiro Dump de Memória com o \$PC=4\$.
4. **Demonstre a Execução:** Digite `s` para iniciar e mostre o avanço do \$PC\$, a movimentação da seta `->` na instrução corrente e o acendimento correspondente dos 4 LEDs na protoboard a cada 4 segundos.